

# Foundations of Cybersecurity/Applied Cryptography

## 13 June 2023

### Exercise n. 1 [12] [All]

1. Illustrate the Diffie-Hellman protocol.
2. Argue about its security w.r.t. a passive and an active adversary.
3. Argue why it is more secure to run the protocol in a sub-group  $H$  of  $\mathbb{Z}_p^*$  rather than in the  $\mathbb{Z}_p^*$  itself. Which feature must  $H$  possess?

### Exercise n. 2 [10] [All]

Consider the following authentication and key agreement protocol:

M1  $A \rightarrow B: \{A, B, n_A, K'\}_{K_{AB}}$

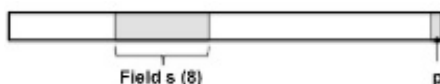
M2  $B \rightarrow A: \{A, B, n_A\}_{K'}$

where  $K_{AB}$  is a long-term key shared by Alice and Bob,  $n_A$  is a nonce, and  $K'$  is the session key.

1. Analyse the protocol and specify the assumptions under which it fulfils the key authentication and key confirmation requirements.
2. Discuss whether  $n_A$  is necessary or if it can be removed from the protocol.
3. Argue whether the protocol is subject to a replay attack.
4. In case of positive answer to the previous question, modify the protocol to remove the vulnerability.

### Exercise n. 3 [8] [2022-23]

Assume that a plaintext  $P$  has the format specified in the figure where, in particular,  $s$  is an 8-bit field that specifies an amount of money and  $p$  is a parity bit s.t.  $p$  is 0 if the number of 1s in the plaintext (bit  $p$  excluded) is even; it is 1 otherwise.



The whole plaintext is encrypted by means of one-time-pad. Answer the following questions.

1. Is this encryption scheme malleable?
2. Assume that field  $s$  specifies the value 130. Argue whether and how, it is possible to modify the cipher-text so that the decrypted plaintext specifies 146 in the field  $s$  and such a modification goes undetected.
3. Propose a possible countermeasure.
- 4.

### Exercise n. 3 [8] [earlier]

Find and explain the vulnerabilities of the following function and patch them.

```
unsigned int do_stuff(unsigned int a, int b){  
    return a<<b;  
}
```

# Solution

## EXERCISE N.1

See theory.

## EXERCISE N.2

**Question 1.** The main assumption is that B believes  $K'$  fresh.

**Question 2.** If A generates fresh session keys, then  $n_A$  may be removed.

**Question 3.** Because of the assumption at point 1), an adversary that manages to obtain a session key  $K'$  then (s)he can impersonate Alice with respect to Bob.

**Question 4.** A possible solution is the following:

M1  $B \rightarrow A: n_B$

M2  $A \rightarrow B: \{A, B, n_B, K'\}_{K_{AB}}$

M3  $B \rightarrow A: \{B, A, n_B + 1\}_{K'}$

## EXERCISE N.3 [2022-23]

**Question 1.** The encryption scheme is malleable. A simple way to prove it is the following. Let  $P[i]$ ,  $C[i]$ , and  $K[i]$  be the  $i$ -th bit of the plaintext, ciphertext and key, respectively, s.t.  $C[i] = P[i] \text{ xor } K[i]$ . Notice that  $P[0] = p$ . Finally let  $C'$  be the modified ciphertext and  $P'$  the resulting plaintext after decryption. Notice that an adversary can easily complement a bit of the plaintext by operating on the ciphertext. Assume that the adversary wishes to complement bit  $i$ . Then, (s)he computes  $C'[i] = C[i] \text{ xor } 1 = (P[i] \text{ xor } K[i]) \text{ xor } 1 = (P[i] \text{ xor } 1) \text{ xor } K[i] = P'[i] \text{ xor } K[i]$ . It follows that  $P'[i] = P[i] \text{ xor } 1$ , that is,  $P'[i]$  is the complement of  $P[i]$  ( $P'[i] = \sim P[i]$ ).

However, for the attack to go undetected, the parity bit must be consistently modified as well. Notice that since  $P[i]$  is complemented the number of 1 either increments or decrements by one. In both cases the parity bit must be complemented as well, that is  $C'[0] = C[0] \text{ xor } 1$ .

**Question 2.** The attack consists in complementing both  $C[0]$  and  $s[4]$ :

$$s' \leftarrow s \oplus 00010000; p' \leftarrow p \oplus 1$$

**Question 3.** The problem can be solved by replacing the parity bit by a tag resulting from a secure hash function.

## EXERCISE N.3 [EARLIER]

This is a case of usual conversions where operation mixes signed and unsigned operands. In this case, the better-ranked-unsigned rule applies and int is converted to uint. The problem appears when the int operand is negative. A possible sanitization is the following:

```
unsigned int do_stuff(unsigned int a, int b){
    if (b < 0) return a << -b;
    return a<<b;
}
```